

Assignment #1: C++ Basics & Classes

Due Date 1: Sunday, October 4, 2026, 5:00 pm EST

Due Date 2: Sunday, October 11, 2026, 5:00 pm EST

Learning objectives:

- C++ I/O (standard, file streams) and output formatting
- C++ Struct
- C++ strings and stringstream
- C++ references, pointers, and dynamic memory allocation

- **Questions 1a, 2a, and 3a are due on Due Date 1; questions 1b, 2b, and 3b are due on Due Date 2.**
- On this and subsequent assignments, you will take responsibility for your own testing. This assignment is designed to get you into the habit of thinking about testing *before* you start writing your program. For each question, you will be given a compiled executable program which is a program representing a solution to each question. You should use these provided executables to help you write your test cases, as they can show you the resultant output for given inputs. If you look at the deliverables and their due dates, you will notice that there is *no* C++ code due on Due Date 1. Instead, you will be asked to submit test suites for C++ programs that you will later submit by Due Date 2.
Test suites will be in a format compatible with that presented in class.
- Design your test suites with care; they are your primary tool for verifying the correctness of your code. Note that test suite submission zip files are restricted to contain a maximum of 40 tests. The size of each input (.in) file is also restricted to 300 bytes, and each output file (.out) is restricted to 1,000 bytes. This is to encourage you not to combine all of your testing in a single pair of files.
- You must use the standard C++ I/O streaming and memory management (MM) facilities on this assignment; you may **not** use C-style I/O or MM. More concretely, you may `import` the following C++ libraries (and no others!) for the current assignment: `iostream`, `fstream`, `sstream`, `iomanip`, and `string`. MarkUs will be setup to **reject** submissions that use C-style I/O or MM, or libraries other than the ones specified above.
- We will manually check that you follow a reasonable standard of documentation and style, and to verify any assignment requirements that are not automatically enforced by MarkUs. Code to a standard that you would expect from someone else if you had to maintain their code. Further comments on coding guidelines can be found here: <https://cmsweb.utoronto.ca/cscb07f26/codingGuidelines/>
- We have provided some code and sample executables in the subdirectory `codeForStudents` under the appropriate subdirectories. **These executables have been compiled on the matlab server and will not run anywhere else.**
- **You may not ask public questions on Piazza about what the programs that make up the assignment are supposed to do.** A major part of this assignment involves designing test cases, and questions that ask what the programs should do in one case, or another will give away potential test cases to the rest of the class. Questions found in violation of this rule will be marked private or deleted; repeat offences could be subject to discipline.
- You may publicly discuss the coding assessment portion of the questions on Piazza as long as solutions are neither discussed nor revealed.

Question 1

1. The five-letter word game is a logic game similar to Hangman and Mastermind. A secret five-letter word with no repeating letters is chosen. A player guesses five-letter words (also with no repeating letters). For each word guessed, the player is informed how many letters in the word that they guessed are in the secret word. When the player guesses the correct word, the game is over, and they win.

An implementation of this game should behave according to the following requirements:

- The program will read a single word (the “secret”) from a file specified on the command line, which is the word that the player should attempt to guess. The syntax for invoking your program should be:

```
wordguess [filename]
```

- If invoked incorrectly (i.e. with an incorrect number of arguments), the program should write a message to standard error stating “usage: wordguess [filename]” and exit.
 - If a file is specified on the command line which cannot be opened for any reason, the program should write a message to standard error stating “file cannot be opened”, where “file” is replaced with the filename specified on the command line, and then exit.
 - If the secret word has repeating letters, or is not exactly five characters in length, the program should write a message to standard error stating “the secret word is invalid” and exit.
 - The game will read a word from standard input (a “guess”), and provide feedback as described below. Players can continue to enter guesses, one word at a time, until the secret word is guessed, or an EOF (Ctrl-D) is entered.
 - A valid word consists of five letters, with no letters repeating in a word (e.g. “poach” is a valid word; “speed” is not). Words do not include digits or symbols, and do not contain spaces (i.e. words containing non-letter symbols are not valid).
 - Guesses should consist of lower-case (e.g. “chase” is a valid guess but “chASe” is not), but you do not need to validate or enforce this in your program.
 - All string comparisons should be case sensitive (i.e. “SPACE”, “space” and “Space” are not the same word). Guesses that differ from the secret only by case of the letter entered are not considered a match (e.g. “claim” doesn’t match “clAim”).
 - If the guess is not equal to five characters in length, or contains duplicate letters, the program should write a message to standard output stating “invalid guess”, and allow the player to guess again.
 - If a guess does not match the secret word exactly, the game will return a message stating that “x letters match” where x denotes the number of characters in that guess that are also in the secret word. Note that position does not matter; guessing “poach” against the secret word “crash” would match 3 characters, the ‘a’, ‘c’ and ‘h’, even though they had different positions in the two words. The player is allowed to guess again.
 - If a word entered matches the secret word, the program will return a message “you guessed correctly!” and exit.
- (a) **Due on Due Date 1:** Design a test suite for this program. List the name of your test in your suite file `suiteq1.txt`. Zip your suite file, together with the associated `.in`, `.out` and `.args` files, into the file `alq1.zip`.
 - (b) **Due on Due Date 2:** Write the program in C++. Call your program `wordguess.cc`.

Question 2

2. We typically use arrays to store collections of items (say, integers). We can allow for limited growth of a collection by allocating more space than typically needed, and then keeping track of how much space was actually used. We can allow for unlimited growth of the array by allocating the array on the heap and resizing as necessary. The following structure encapsulates a partially-filled array:

```
struct IntArray {
    int size; // number of elements the array currently holds
    int capacity; // number of elements the array could hold, given current
                // memory allocation to contents
    int *contents;
};
```

- Write the function `readIntArray` which returns an `IntArray` structure, and whose signature is as follows:

```
IntArray readIntArray();
```

The function `readIntArray` consumes as many integers from `cin` as are available, populates an `IntArray` structure in order with these, and then returns the structure. If a token that cannot be parsed as an integer is encountered before the structure is full, then `readIntArray` fills as much of the array as needed, leaving the rest unfilled. If a non-integer is encountered, the first offending character should be removed from the input stream (i.e., call `cin.ignore` once with no arguments). In all circumstances, the field `size` should accurately represent the number of elements actually stored in the array and `capacity` should represent the amount of storage currently allocated to the array.

- Write the function `addToIntArray`, which takes a reference to an `IntArray` structure and adds as many integers to the structure as are available on `cin`. The behaviour is identical to `readIntArray`, except that integers are being added to the end of an existing `IntArray`. The signature is as follows:

```
void addToIntArray(IntArray&);
```

- Write the function `printIntArray`, which takes a reference to an `IntArray` structure, and whose signature is as follows:

```
void printIntArray(const IntArray&);
```

The function `printIntArray(a)` prints the contents of `a` (as many elements as are actually present) to `cout`, on the same line, separated by spaces, and followed by a newline. There should be a space after each element in the array (including the last element), and not before the first element.

It is not valid to print or add to an array that has not previously been read, because its fields may not be properly set. You should not test this.

For memory allocation, you **must** follow this allocation scheme: every `IntArray` structure begins with a capacity of 0. The first time data is stored in an `IntArray` structure, it is given a capacity of 5 and space allocated accordingly. If at any point, this capacity proves to be not enough, you must double the capacity (so capacities will go from 5 to 10 to 20 to 40 ...). Note that there is no `realloc` in C++, so doubling the size of an array necessitates allocating a new array and copying items over. Your program must not leak memory.

A test harness is available in the starter file `alq2.cc`, which you will find in your `cscb07f26/a1` directory. **Make sure you read and understand this test harness, as you will need to know what it does in order to structure your test suite.** Note that we may use a different test harness to evaluate the code you submit on Due Date 2 (if your functions work properly, it should not matter what test harness we use).

- Due on Due Date 1:** Design a test suite for this program, using the main function provided in the test harness. Call your suite file `suiteq2.txt`. Zip your suite file, together with the associated `.in` and `.out` files, into the file `alq2.zip`.
- Due on Due Date 2:** Define the functions listed above in the starter file `alq2.cc`.

Question 3

3. In this exercise, you will write a C++ class (implemented as a `struct`) that adds support for *rational numbers* to C++. In mathematics, a rational number is any number that can be expressed as a fraction n/d of two integers, a numerator n and a non-zero denominator d . Since d could be equal to 1, every integer is also a rational number. In our `Rational` class, rational numbers are always stored in their most simplified form, e.g. $4/8$ is stored as $1/2$, $18/8$ as $9/4$. Additionally, negative rational numbers are stored with a negative numerator and a positive denominator, e.g. negative one-quarter is stored as $-1/4$ and not $1/-4$.

An interface file (`rational.cc`) containing all the methods and functions to implement has been provided in the `cscb07f26/a1` directory. Implement the required methods and functions in a file called `rational-impl.cc`.

Implement a two-parameter constructor with the following signature:

```
Rational(int num = 0, int den = 1);
```

To support arithmetic for rational numbers, overload the binary operators `+`, `-`, `*` and `/` to operate on two rational numbers. In case you have forgotten how these operations work, here is a refresher:

$$\frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{bd} \qquad \frac{a}{b} - \frac{c}{d} = \frac{ad-bc}{bd} \qquad \frac{a}{b} * \frac{c}{d} = \frac{ac}{bd} \qquad \frac{a}{b} / \frac{c}{d} = \frac{ad}{bc}$$

Also, implement the following:

- A convenience unary `-` operator, which negates the rational number.
- Convenience `+=`, `-=` operators where `a += b` has the effect of setting `a` to `a + b` (similarly for `-=`)
- The helper method `simplify()`, which can be used to update a rational number to its simplest form.
- Accessor methods `getNumerator()` and `getDenominator()` that return the numerator and denominator of the rational number, respectively.
- Method `isZero()` returns true if the rational number is zero.
- Implement the overloaded input operator for rational numbers as the function:

```
std::istream &operator>>(std::istream &, Rational &);
```

The format for reading rational numbers is: an int-value-for-numerator followed by the `/` character followed by an int-value-for-denominator. Arbitrary amounts of whitespace are permitted before or in between any of these terms. A denominator must be provided for all values even if the denominator is 1 (this includes the rational number 0) e.g. the rational number 5 must be input as `5/1`.
- Implement the overloaded output operator for rational numbers as the function:

```
std::ostream &operator<<(std::ostream &, const Rational &);
```

The output format is the numerator followed by the `/` character and then the denominator without any whitespace. Rational numbers that have a denominator of 1 are printed as integers e.g. `17/1` is printed as `17`.

A test harness is available in the file `alq3.cc`, which you will find in your `cscb07f26/a1` directory. **Make sure you read and understand this test harness, as you will need to know what it does in order to structure your test suite.** Note that we may use a different test harness to evaluate the code you submit on Due Date 2 (if your functions work properly, it should not matter what test harness we use). You should not modify `alq3.cc`.

- Due on Due Date 1:** Design a test suite for this program. Call your suite file `suiteq3.txt`. Zip your suite file, together with the associated `.in`, and `.out` files (this program does not accept command line arguments, so there will be no `.args` files), into the file `alq3.zip`.
- Due on Due Date 2:** Submit the file `rational-impl.cc` containing the implementation of methods and functions declared in `rational.cc`.

Submission

The following files are due at Due Date 1: `alq1.zip`, `alq2.zip`, `alq3.zip`.

The following files are due at Due Date 2: `wordguess.cc`, `alq2.cc`, `rational-impl.cc`.